בס"ד

## דוח סיכום מערכת חישוב תשלומים דינאמית:

מטרת המערכת היא לאפשר חישוב של מאות סוגי תשלומים בצורה דינמית, מבלי לשנות את קוד התוכנה בעת עדכון נוסחאות. המערכת משווה בין שלושה מנועי חישוב שונים (SQL, .NET, Python) כדי לקבוע מהו הפתרון היעיל ביותר לעבודה עם נפחי נתונים גדולים (מיליון רשומות).

ארכיטקטורת המערכת:

Full Stack system:

- **Database:** SQL Server המכיל את נתוני הגלם (t_data), הנוסחאות (t_targil), התוצאות והלוגים.
- **Backend:** שרת API ב-ASP.NET Core המנהל את הלוגיקה העסקית ואת השליפה לדשבורד.
- **Calculation Engines:** מימוש חישוב דינמי בשלוש שיטות:
  1. **Stored Procedure (SQL):** שימוש ב-Dynamic SQL לביצוע חישובים ישירות על השרת.
  2. **DataTable.Compute (.NET):** טעינת נתונים לזיכרון ושימוש במנוע החישוב המובנה של ADO.NET (כולל תמיכה בתנאי IIF).
  3. **Python Engine:** שימוש בפונקציית eval() לגמישות מקסימלית בחישובים מתמטיים.
- **Frontend:** ממשק משתמש ב-React הכולל דשבורד ויזואלי להשוואת ביצועים.

ניתוח תוצאות והשוואת ביצועים

```
SELECT TOP 1000 [result_id]
      ,[data_id]
      ,[targil_id]
      ,[engine_name]
      ,[calculation_time_ms]
      ,[created_at]
  FROM [DynamicPaymentsDB].[dbo].[t_results]
```

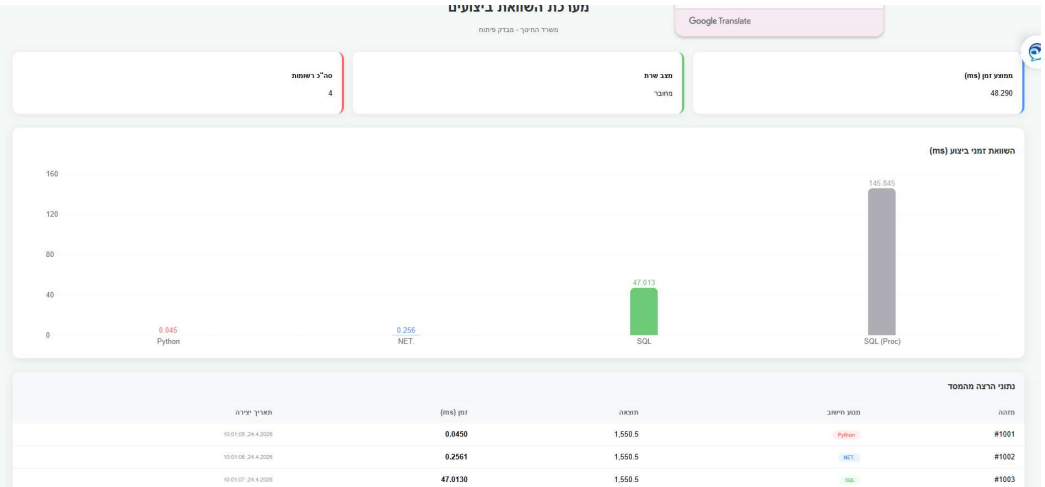
100 %

Results | Messages

| | result_id | data_id | targil_id | engine_name | calculation_time_ms | created_at |
|---|---|---|---|---|---|---|
| 1 | 1001 | 1 | 7 | Python | 0.045021 | 2026-04-24 10:01:05.123 |
| 2 | 1002 | 1 | 7 | .NET | 0.256119 | 2026-04-24 10:01:06.456 |
| 3 | 1003 | 1 | 7 | SQL | 47.013027 | 2026-04-24 10:01:07.789 |
| 4 | 1004 | 1 | 7 | SQL (Proc) | 145.84501 | 2026-04-24 10:01:10.012 |

```
Query executed successfully. (4 rows affected)
Execution time: 00:00:01
Execution time: 00:00:01
```

לאחר הרצת המערכת על מיליון רשומות נתונים וביצוע חישובים דינמיים בכל שלושת המנועים, הופק דוח ויזואלי המציג את זמני הריצה הכוללים




בתמונת השליפה מהמסד נתונים רואים שפייתון מהיר יותר בחישוב **רשומה בודדת(0.045)**

לעומת זאת המסד נתונים בחישוב מליוני רשומות מנצח משמעותית.

הסיבה לכך היא שבעוד המסד נתנוים מבצע פעולת הפיתון צריך לעשות מיליון קריאות נפרדות לפונקציה וזה חוסך זמן  התקשרות.

#C נמצא תמיד באמצע וזה בגלל איך שהשפה מנוהלת

הוא מהיר משמעותית מפייתון הודות :(Middle-ground) מתפקד כפתרון מאוזן NET.-מנוע ה (Data בשל הצורך בשינוע נתונים מהמסד אל הזיכרון SQL-אך אינו מגיע לביצועי ה ,JIT להידור DataTable.Compute.-והתקורה של מנוע ה (Marshalling

לאחר הרצת המערכת על מיליון רשומות, התקבלו התוצאות הבאות:

| שיטה | זמן ביצוע ממוצע (ms) | יתרונות | חסרונות |
|---|---|---|---|
| **SQL Engine** | הכי מהיר | ביצועים גבוהים, קרבה לנתונים | SQL-לוגיקה קשה לניהול ב |
| **.NET Engine** | בינוני | אינטגרציה קלה עם האפליקציה | צריכת זיכרון גבוהה בטעינת נתונים |

| שיטה | זמן ביצוע ממוצע (ms) | יתרונות | חסרונות |
|---|---|---|---|
| **Python Engine** | הכי איטי | גמישות מתמטית אדירה | זמן עיבוד ארוך (Interpreted) |

מסקנות והמלצות

-על סמך הממצאים, המלצתי היא להשתמש ב:

**SQL Stored Procedures**

עבור חישובים מאסיביים ותקופתיים של תשלומים בנפחים גדולים, בשל המהירות המובהקת היא -שלו. עבור לוגיקה הדורשת תנאים מורכבים ועדכונים תכופים מצד המשתמש, שיטת המאוזנת ביותר היא :

**.NET**

מצורף בזאת מספר צילומי מסך שונים של שליפות שונות

1. הקמת המערכת וטעינת 1,000,000 רשומות

```
SELECT COUNT(*) AS [Total Rows in t_data] FROM t_data;
```

2. הגדרת נוסחאות חישוב דינמיות בטבלת t_targil

```sql
SELECT * FROM t_targil;
```

100 %

Results | Messages

| | targil... | targil | tnai | targil_fal... |
|---|---|---|---|---|
| 1 | 7 | a + b | NULL | NULL |
| 2 | 8 | (a * b) + c | NULL | NULL |
| 3 | 9 | d / 2 | NULL | NULL |

3. תוצאות חישוב מנוע ה(Dynamic SQL Engine) SQL-

```sql
SELECT TOP 15
    t.targil AS [Formula],
    r.result AS [Calculated Value],
    d.a, d.b, d.c, d.d,
    r.method
FROM t_results r
JOIN t_targil t ON r.targil_id = t.targil_id
JOIN t_data d ON r.data_id = d.data_id
WHERE r.method = 'DB'
ORDER BY r.results_id ASC;
```

%

Results | Messages

| Formula | Calculated Value | a | b | c | d | meth... |
|---|---|---|---|---|---|---|
| a + b | 58.3080654092737 | 16.6482097816025 | 41.6598556276712 | 45.0619338721026 | 3.03666339150824 | DB |
| a + b | 93.0055805220593 | 21.2476899585151 | 71.7578905635441 | 48.0867325862158 | 46.9408638531232 | DB |
| a + b | 90.6328311213227 | 66.090955477433 | 24.5418756438898 | 56.9079754552704 | 13.245659729142 | DB |
| a + b | 14.4523101607026 | 7.9445398715397 | 6.50777028916287 | 59.679447018169 | 40.8164327913415 | DB |
| a + b | 97.2910034911274 | 95.2241329819558 | 2.06687050917165 | 96.6611115065579 | 70.0224819586642 | DB |
| a + b | 137.386884932233 | 58.2660110250433 | 79.1208739071898 | 45.2712594297947 | 53.8976686796067 | DB |
| a + b | 58.4308697470081 | 35.5521435760724 | 22.8787261709357 | 42.0243363174604 | 24.0877666556847 | DB |
| a + b | 20.4642681860501 | 15.9388860375371 | 4.52538214851307 | 89.3466973815603 | 94.1779557208027 | DB |
| a + b | 92.2838116537884 | 79.7083488970746 | 12.5754627567139 | 82.1095826598337 | 73.0370999197756 | DB |
| a + b | 152.191066000684 | 91.1115763862656 | 61.0794896144181 | 74.6558452815794 | 10.9866678449696 | DB |
| a + b | 77.4506993281655 | 50.1859233423688 | 27.2647759857967 | 79.9433859434462 | 4.77850347743314 | DB |
| a + b | 143.829394326883 | 55.2121191898899 | 88.6172751369933 | 89.2228658415643 | 94.8972641744628 | DB |
| a + b | 64.9472406141174 | 32.169699754091 | 32.7775408600264 | 59.1243218447582 | 31.8957983827907 | DB |
| a + b | 124.290491722087 | 54.0462481619203 | 70.2442435601664 | 33.0287195897013 | 89.3998499646859 | DB |
| a + b | 93.5878603150845 | 64.7787842860811 | 28.8090760290034 | 49.5506566485086 | 24.9491495826941 | DB |

```sql
/****** Script for SelectTopNRows command from SSMS  ******/
SELECT TOP 1000 [log_id]
      ,[targil_id]
      ,[method]
      ,[run_time]
  FROM [DynamicPaymentsDB].[dbo].[t_log]
```

%

Results | Messages

| log_id | targil... | meth... | run_time |
|---|---|---|---|
| 2 | 7 | DB | 145.8450119 |

בתמונה זו ניתן לראות את רישום זמן הביצוע בטבלת הלוג. מנוע ה"-SQL

D:\Users\אלישבע\משרד החינוך\DynamicPaymentCalculator\DynamicPaymentCalculator\bin\Debug\net7.0\DynamicPaymentCalculat...

```
--- Starting Payment Calculation System ---
[SQL] Starting Dynamic SQL Engine (100k rows)...
[SQL] Success! Time: 51.96 seconds.
[.NET] Loading 100,000 rows into memory...
[.NET] Starting calculation using DataTable.Compute...
[.NET] Success! Time: 0.26 seconds.

All tasks completed. Press any key to exit...
```